

```
public class UserData {
    private String mFirstName;
    private String mLastName;

    public UserData(String firstName, String lastName) {
        mFirstName = firstName;
        mLastName = lastName;
    }
    ..
    ..
    //getters and setters
}
```

This is a very basic model. Let's take a look at the layout file for our activity.

```
<layout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools">

    <data>
        <variable
            name="user"
            type="in.samvidinfotech.bindingdemo.UserData" />
    </data>

    <LinearLayout
        android:orientation="vertical"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:gravity="center">

        <TextView
            android:text="@{user.firstName}"
            tools:text="First Name"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"/>

        <TextView
            tools:text="Last Name"
            android:text="@{user.lastName}"
            android:layout_marginTop="10dp"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"/>

    </LinearLayout>
</layout>
```

Our layout file is where all the magic happens. The essence of the data binding framework on Android is that it lets us put expressions in our XML code. Now, to indicate that a layout file is using data binding, we have the `<layout>` tag at the top of our view hierarchy. The layout inside *LinearLayout* is our normal activity layout. The `<data>` tag contains information about the data being used

inside the current XML file. So, here we declare a variable in the `<variable>` tag. The name attribute defines the name to be used in the layout file to refer to the variable, and the type attribute determines the type of the class. We can have many variables inside a single layout file.

Now let's look at the code of activity to set the data in the layout file:

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    ActivityUserBinding activityUserBinding = DataBindingUtil.
        setContentView(this, R.layout.activity_user);

    activityUserBinding.setUser(new UserData("Test", "User"));
}
```

Pretty neat, right? The code to set the data in our layout file is only two lines. Now, where did the *ActivityUserBinding* class come from? The data binding framework generates compile time classes based on our layout files, which lets the data binding framework work without using any reflection at runtime. The name of these classes is given by converting the name of the layout file to 'camel case' and appending binding to it. For instance, the layout name *activity_user.xml* becomes *ActivityUserBinding*. These classes live in the data binding package inside your application, i.e., `<your-package-name>.databinding.ActivityUserBinding`. You can also change the place where the classes are stored.

Writing observable models

Currently, if you run your application, it will set the data in the activity, but it won't update the data in the UI if the data in our model changes. In order to do that, we need to make our models smarter — we need to make them observable. The easiest way to do that would be to extend the *BaseObservable* class; it provides convenience methods for notifying you about the data changes. Now, if we change our *UserData* model to be an observable model, it will look like what's shown below:

```
public class UserData_BaseObservable extends BaseObservable {
    @Bindable
    private String mFirstName;
    @Bindable
    private String mLastName;

    public UserData_BaseObservable(String firstName, String
        lastName) {
        mFirstName = firstName;
        mLastName = lastName;
    }

    public String getFirstName() {
        return mFirstName;
    }
}
```